

双指针：一般要满足单调性^[OJ]

在正整数的前提下，采用滑动窗口的方法来使它像蜗牛蠕动一样得出答案：^[OJ]

209. 长度最小的子数组

已解答 

中等

 相关标签

 相关企业

Aa

给定一个含有 n 个正整数的数组和一个正整数 `target` 。

找出该数组中满足其总和大于等于 `target` 的长度最小的 **子数组** `[numsl, numsl+1, ..., numsr-1, numsr]`，并返回其长度。如果不存在符合条件的子数组，返回 `0`。

示例 1：

输入：target = 7, nums = [2,3,1,2,4,3]

输出：2

解释：子数组 `[4,3]` 是该条件下的长度最小的子数组。

示例 2：

输入：target = 4, nums = [1,4,4]

输出：1

示例 3：

输入：target = 11, nums = [1,1,1,1,1,1,1,1]

输出：0

提示：

- `1 <= target <= 109`
- `1 <= nums.length <= 105`
- `1 <= nums[i] <= 104`

进阶：

- 如果你已经实现 `O(n)` 时间复杂度的解法，请尝试设计一个 `O(n log(n))` 时间复杂度的解法。

```

class Solution(object):
    def minSubArrayLen(self, target, nums):
        """
        :type target: int
        :type nums: List[int]
        :rtype: int
        """
        n=len(nums)
        s= 0
        ans = n +1
        left = 0
        for right, x in enumerate(nums):
            s += x
            while s - nums[left] \>= target:
                s -= nums[left]
                left += 1
            if s \>= target:
                ans = min(ans, right - left +1)
        return ans if ans \<= n else 0

```

或者直接针对答案: `OBJ OBJ`

```

class Solution(object):
    def minSubArrayLen(self, target, nums):
        """
        :type target: int
        :type nums: List[int]
        :rtype: int
        """
        n=len(nums)
        s= 0
        ans = n +1
        left = 0
        for right, x in enumerate(nums):
            s += x
            # while s - nums[left] \>= target:
            #     s -= nums[left]
            #     left += 1
            # if s \>= target:
            #     ans = min(ans, right - left +1)
            while s \>= target:
                ans = min(right-left+1, ans)
                s -= nums[left]

```

```
        left += 1
    return ans if ans <= n else 0
```

713. 乘积小于 K 的子数组

已解答 

中等

 相关标签

 相关企业

 提示

Ax

给你一个整数数组 `nums` 和一个整数 `k`，请你返回子数组内所有元素的乘积严格小于 `k` 的连续子数组的数目。

示例 1:

输入: `nums = [10,5,2,6]`, `k = 100`

输出: 8

解释: 8 个乘积小于 100 的子数组分别为:

`[10]`、`[5]`、`[2]`、`[6]`、`[10,5]`、`[5,2]`、`[2,6]`、`[5,2,6]`。

需要注意的是 `[10,5,2]` 并不是乘积小于 100 的子数组。

示例 2:

输入: `nums = [1,2,3]`, `k = 0`

输出: 0

提示:

- `1 <= nums.length <= 3 * 104`
- `1 <= nums[i] <= 1000`
- `0 <= k <= 106`

```
class Solution(object):
    def numSubarrayProductLessThanK(self, nums, k):
        """
        :type nums: List[int]
        :type k: int
```

```

:rtype: int
"""
if k <=1:
    return 0
ans = 0
prod = 1
left = 0
for right, x in enumerate(nums):
    prod *=x
    while prod >= k:
        prod /= nums[left]
        left += 1
    ans += right - left +1
return ans

```

关键就在于：

当窗口经过 while prod >= k 调整完以后，区间 [left, right] 已经满足：

nums[left..right] 的乘积 < k

而且因为 left 已经是当前这个 right 下，最靠左还能成立的位置，所以：

以 right 作为结尾、乘积小于 k 的所有子数组，恰好就是：

- [right, right]
- [right-1, right]
- [right-2, right]
- ...
- [left, right]

一共有 right - left + 1 个。

这就是为什么要写：

ans += right - left + 1需要额外注意的是这里的右端点是固定的，也就是我们都是以右端点为参照：

假设某一步：

nums = [10,5,2,6]

0 1 2 3

当：

left = 1

right = 3

窗口是：

[5,2,6]

窗口长度：

$$3 - 1 + 1 = 3$$

此时统计的是 **所有以 3 结尾的子数组**：

[6] 长度1

[2,6] 长度2

[5,2,6] 长度3

一共 **3 个**。

一、滑动窗口解决的核心问题

滑动窗口只解决一种问题：

连续子数组 / 子串问题

形式通常是：

- 最长子串
- 最短子串
- 子数组数量
- 满足某种条件的区间

关键特点：

数组或字符串

要求 连续

二、统一思维模型

窗口就是一个区间：

[left right]

窗口始终表示当前正在考虑的子数组。

每一步：

1 右端点扩展窗口

2 如果不合法，左端点收缩

3 统计答案

流程永远是：

right 扩张

left 收缩

更新答案

三、最通用模板

这是所有滑动窗口题的骨架：

left = 0

for right in range(len(nums)):

1 扩张窗口

```
add(nums[right])
```

2 如果窗口不合法，缩小

```
while window_invalid:
```

```
remove(nums[left])
```

```
left += 1
```

3 更新答案

```
update_result()
```

三个函数其实就是你自己写的逻辑：

```
add()
```

```
remove()
```

```
window_invalid
```

四、四种最常见题型

滑动窗口基本只变这四种。

理解它们就够了。

1 最长子串问题

例题：

Longest Substring Without Repeating Characters

特点：

窗口必须一直保持合法

模板：

```
left = 0
```

```
for right in range(n):
```

```
add(nums[right])
```

```
while 窗口不合法:
```

```
remove(nums[left])
```

```
left += 1
```

```
ans = max(ans, right - left + 1)
```

例子：

- 无重复字符子串
- 最多 K 个不同字符

2 最短子串问题

例题：

Minimum Window Substring

特点:

窗口先扩展到满足条件, 再缩小

模板:

```
left = 0
```

```
for right in range(n):
```

```
    add(nums[right])
```

```
    while 窗口满足条件:
```

```
        ans = min(ans, right - left + 1)
```

```
        remove(nums[left])
```

```
        left += 1
```

注意这里是:

```
while valid
```

不是 invalid

3 统计子数组数量

例题:

Subarray Product < K

模板:

```
left = 0
```

```
for right in range(n):
```

```
    add(nums[right])
```

```
    while 窗口不合法:
```

```
        remove(nums[left])
```

```
        left += 1
```

```
ans += right - left + 1
```

这里是统计:

所有以 right 结尾的合法子数组

4 固定长度窗口

例题:

Maximum Average Subarray

模板:

```
window = 0
```

```
k = 固定长度
```

```
for right in range(n):
```

```
    window += nums[right]
```

```
    if right >= k:
```

```
        window -= nums[right-k]
```

```
    if right >= k-1:
```

update answer

这种最简单。

五、滑动窗口三条铁律

记住这三条，基本不会写错。

第一条

右端加入谁，就 add 谁

add(nums[right])

第二条

左端移出谁，就 remove 谁

remove(nums[left])

left += 1

第三条

窗口长度

right - left + 1

永远这么算。

六、什么时候用 HashMap

一般是这三种情况：

字符出现次数

Counter()

字符最后出现位置

dict()

字符是否存在

set()

七、判断是否是滑动窗口题

看到这些关键词几乎就是：

- substring
- subarray
- longest
- shortest
- continuous
- at most k
- without repeating

八、为什么滑动窗口是 $O(n)$

因为：

right 只往右走一次

left 只往右走一次

每个元素最多被：

加入窗口一次

移出窗口一次

所以复杂度：

$O(n)$